BetterCode: Reliable

Your code is wrong, so design for that

Overview

- *Recap day 1+2+3* - readable, reusable, reproducible
  - Help with code comprehension & abstraction & execution
  - Efficient repeatable analyses
- **Reliable**: Revision
  - Help with confidence
  - Activity 1: Edge cases
  - Activity 2: Unit tests

Reliable science: peer-review process

What is reviewed carefully	**What determines correctness**
Writing clarity	*Code logic*
Figures	*Data processing*
Conclusions	*Implementation details*
Narrative	*Code quality*
Format / Aesthetics	*Code documentation*

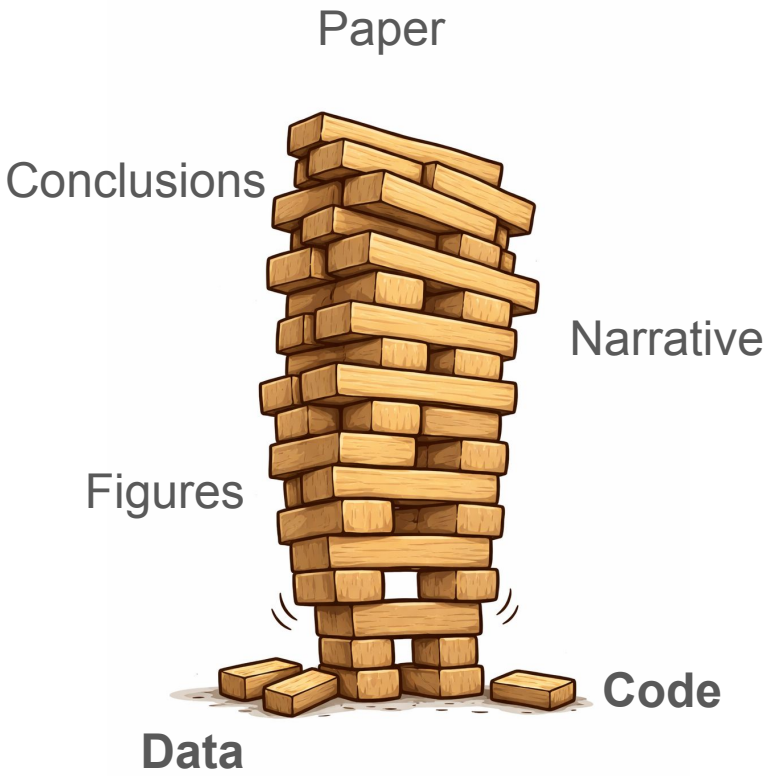
Time spent in research project:

- Writing & figures: [redacted]
- Analysis coding: [redacted]
- Code review/testing: [redacted]

Why should code be reliable?

Code errors can be “silent”

Incorrect code → incorrect science



What makes code reliable? Tried and true

- Readable, reusable & reproducible
- Correct - right output for the right reasons
- Robust - handle edge cases
- Reviewed

Demo 1a: Silent break - Boxplot

Visualize mean measurement
among different groups using boxplots.

Where is the code “wrong”?

Demo 1b: Silent break - Moving average

I’m smoothing out time-series data
using moving average.

Where is the code “wrong”?

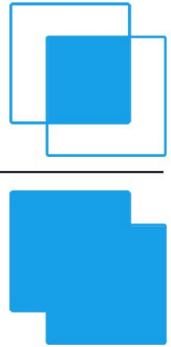
Demo 1: Silent break

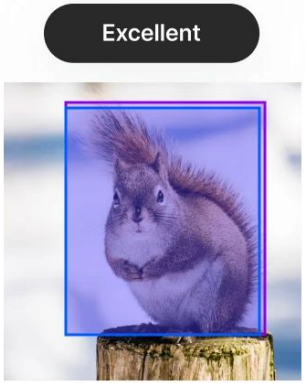
Suggestions:

- Export & inspect intermediate data, *not just* inputs → outputs
- Check distribution & sample individual data points, *not just* summary statistics
- What condition may break the code, not just this particular dataset/settings

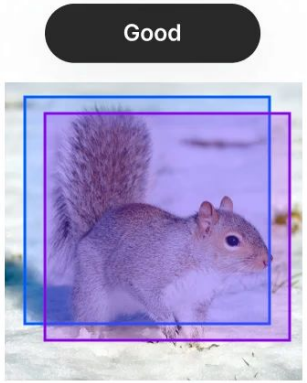
Activity 1 - Identify potential edge cases

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

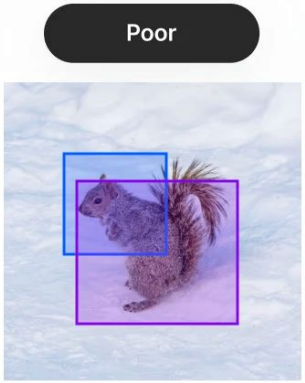




IoU= 0.95



IoU= 0.79



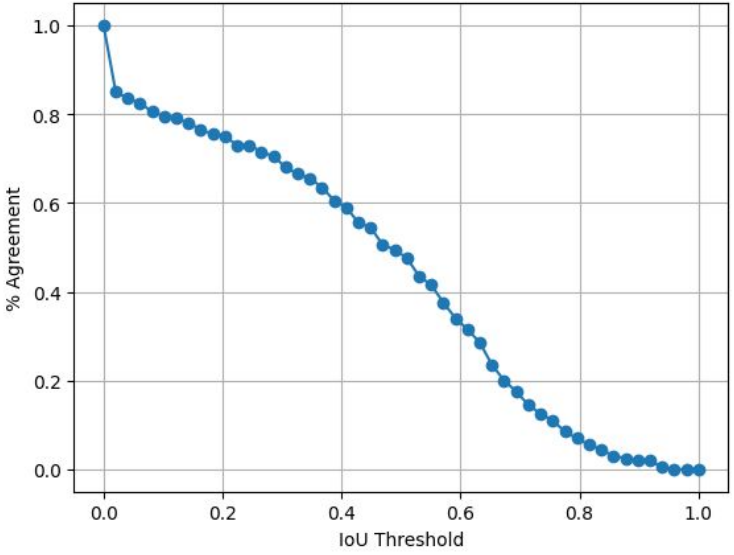
IoU= 0.45

Activity 1 - Identify potential edge cases

Two annotators segmented 200 images of cells. We compute IoU to compare the similarity of the two annotation sets.

What (potential) issues does the code have?

How can we fix it?



Activity 1 - Discussion

Which potential problems (edge cases) are easy to think of? Which are hard?

How do you check its correctness?

(Software) Code Testing

Smoke	Does it run at all?
Unit	Is each function correct?
Integration	Do components work together?
System	Does the whole pipeline work?
Acceptance	Does it solve the real problems?
Regression	Do old results still hold?

Reference: https://book.the-turing-way.org/reproducible-research/testing/testing-overview/

(Data Analysis) Code Testing

Software Code	Data Analysis Code
Pipeline-centric	Data-centric
Clear inputs → outputs	Less clear outputs
Generalizable	Implicit assumptions
Unit tests	Sanity + validation

Good habits:

- Adopt *appropriate* software code testing
- QC checkpoints
  - Same data, different figures
  - Outliers
- Peer discussion
  - Small code chunks
  - Intermediate/QC outputs

Demo 2: Unit test with pytest / testthat

Write unittest for the demo 1a.

Potential options:

- Input structure: empty, singleton, multiple
- Error handling
- Mock cases: expected output/dimensions

Activity 2 - Write your unit test

Write 2 unit tests for the code in activity 1.

- One test for a typical input
- One test for an edge case

Take-away message

- Don’t trust your results until you’ve reviewed your code at multiple stages and from multiple angles